

Lab 3: Architecture Justification

Blood Bank Inventory & Emergency Donor Matcher (Problem Statement #12)

Name: Harish Ramesh Kumar SRN: PES1UG24AM111 Section: B Date: 02 October 2026

Architecture Selection

We chose **Microservices Architecture** for the **Blood Bank Inventory & Emergency Donor Matcher** System.

Architectural Choice

The system is split into five independently deployable services, Emergency Request, Inventory, Donor Matching, Donor Profile and Notification, plus the external SMS Gateway from Lab 1. Each service owns its own data store and offers provided interfaces that other services call through API calls. The services map onto the four Lab 2 Epics (Epic 2 is split into Donor Matching and Notification) and together cover all 14 User Stories. The three Lab 1 actors use them through provided interfaces: the Emergency Requester through `IEmergencyRequest`, the Blood Bank Manager through `IStockManagement`, and the Donor through `IDonorRegistration`. The diagram follows the UC-01 flow: check inventory, match donors, look up profiles, send SMS alerts, record Confirm/Decline replies and update staff.

Reason 1: Different load and failure profiles

FR-001 requires SMS alerts to compatible donors within 30 seconds of an emergency flag, so alert traffic comes in bursts, while stock updates (FR-003) are small and steady. The Donor Matching and Notification services can be scaled on their own, and a slow or failed SMS Gateway only delays alerts; it cannot block stock updates or request intake. This also suits Alternate Flow 7a of UC-01: a donor who does not respond within 15 minutes is marked "No Response" inside Donor Matching, without blocking the rest of the flow.

Reason 2: Consistency-critical and sensitive data kept in one owner

NFR-001 (Story 3.4, Prevent Double Allocation) requires that the same blood bag is never allocated twice. Only the Inventory Service reads and writes the stock ledger, and the Request Service reserves units through `IInventoryCheck`, so the rule is enforced in one place. Likewise, donor profiles live only in the Donor Profile Service, which keeps NFR-002 (Story 4.2, Encrypt Donor Data) in one place. The split also matches the Lab 2 sprint order, where inventory, donor profiles and compatibility logic were built before alerts and escalation.

Security Advantage

Donor data (blood group, contact number, location) is stored only in the Donor Profile Service and reached only through the `IDonorLookup` interface, so no other service has direct database access. Data is protected with TLS 1.2+ in transit and AES-256 at rest (NFR-002). The Emergency Request Service accepts requests only from verified medical centres (FR-004). If one service such as Notification is compromised, the damage is limited because it does not store donor profiles.

Performance Benefit

Alerts go through `INotifyDonors` and `INotifyStaff` to a separate Notification Service, so the Request Service can respond immediately while alerts are sent in parallel, which supports the 30-second target of FR-001. The Inventory Service also runs on its own, so a stock change stays visible on the manager dashboard within 2 seconds (FR-003) and expiry flags (FR-005) stay current even while an alert burst is running.

Why not Layered or Client-Server?

A Layered design would deploy and scale everything as one unit, so an alert surge would slow inventory updates. A centralised Client-Server design has a single point of failure and a scalability bottleneck, which is unacceptable when delays can affect a patient. The cost of Microservices (network latency, operational complexity, data consistency across services) is managed by keeping the service count small (five) and keeping the inventory ledger in a single service.